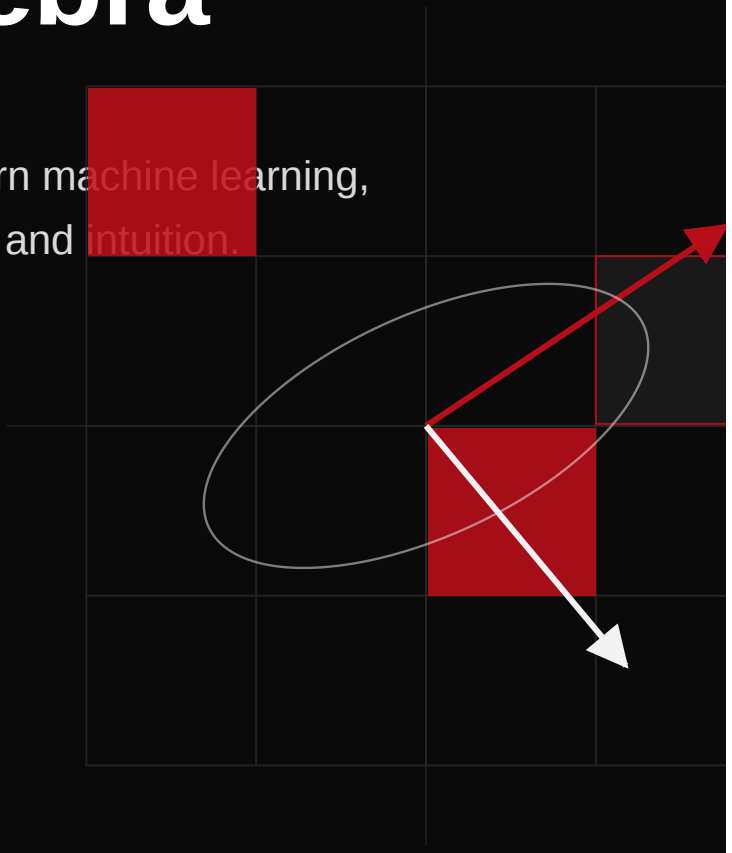
A DATA SCIENTIST'S COMPANION

Introduction to Linear Algebra

The mathematics behind modern machine learning, explained with diagrams, code, and intuition.



By **M. Aammar Amjad**

First Edition · April 2026

Preface

This book is a compact, no-nonsense guide to linear algebra written for people who want to *use* it — most of all, data scientists and machine-learning engineers. Every chapter follows the same rhythm: concise definitions, one clean worked example, a diagram, a short NumPy snippet, one multiple-choice question, one or two problems to solve, and a one-glance recap card.

The goal is not rigour for rigour's sake. The goal is **fluency**: when you read a paper that says "we project onto the top-k singular vectors" or "the Hessian is positive definite", you should picture what is happening.

HOW TO READ THIS BOOK

1. Skim the diagram first — the picture carries 60% of the idea.
2. Read the definition box. Then the worked example.
3. Answer the MCQ *before* peeking at the answer.
4. Attempt the problems with pen and paper, then verify in NumPy.
5. Close the chapter by re-reading only the recap card.

Notation is kept standard: bold lower-case for vectors ($\mathbf{v}$), upper-case for matrices (A), $\mathbb{R}^n$ for n-dimensional real space, $A^\top$ for transpose, A^{-1} for inverse, and $\langle \mathbf{u}, \mathbf{v} \rangle$ for the inner product.

— M. Aammar Amjad

Index

1. Vectors & Vector Spaces 1

- Scalars, vectors, arithmetic
- Dot product & norms
- Geometry of vectors

2. Matrices & Matrix Operations 8

- Matrix algebra
- Matrix–vector products
- Special matrices

3. Systems of Linear Equations 15

- Row-echelon form
- Gaussian elimination
- Existence & uniqueness

4. Determinants & Invertibility 22

- Cofactor expansion

- Properties of determinants
- Invertibility theorem

5. Basis, Dimension & Rank 28

- Linear independence
- Four fundamental subspaces
- Rank–nullity

6. Linear Transformations 35

- Matrix of a transformation
- Change of basis
- Kernel & image

7. Orthogonality, Projections & Least Squares 42

- Inner product & orthogonality
- Gram–Schmidt
- Normal equations & regression

8. Eigenvalues & Eigenvectors 50

- Characteristic polynomial
- Diagonalisation
- Symmetric matrices

9. Singular Value Decomposition & PCA 58

- The SVD theorem
- Low-rank approximation
- Principal Component Analysis

10. Matrix Factorisations 66

- LU decomposition
- QR decomposition
- Cholesky decomposition

11. Numerical Linear Algebra 72

- Conditioning & stability
- Sparse matrices
- Iterative solvers

12. Applications in Data Science 80

- Word embeddings & PageRank
- Recommender systems
- Neural network internals

End matter

A. Answer Key

B. NumPy Cheatsheet

C. Glossary

D. References & Further Reading

01 CHAPTER ONE · THE BUILDING BLOCK

Vectors & Vector Spaces

A **vector** is the simplest object in linear algebra — an ordered tuple of numbers. For a data scientist it is everything: a single row of a dataset, an image flattened into pixels, a document's TF–IDF representation, a gradient during training.

1.1 Scalars, vectors, and arithmetic

DEFINITION · VECTOR IN $\mathbb{R}^N$

A vector $\mathbf{v} \in \mathbb{R}^n$ is an ordered tuple $\mathbf{v} = (v_1, v_2, \ldots, v_n)^\top$ where each $v_i \in \mathbb{R}$. We add vectors component-wise and scale them by a scalar $\alpha \in \mathbb{R}$:

$$\mathbf{u} + \mathbf{v} = (u_1 + v_1, \ldots, u_n + v_n)^\top, \qquad \alpha\mathbf{v} = (\alpha v_1, \ldots, \alpha v_n)^\top.$$

Addition is commutative and associative, scaling distributes over addition, and the zero vector $\mathbf{0}$ is the identity. Together, these 8 axioms define a *vector space*.

1.2 Dot product and norm

The **dot product** measures how aligned two vectors are:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{n} u_i v_i = \|\mathbf{u}\| \, \|\mathbf{v}\| \, \cos\theta. \tag{1.1}$$

The **Euclidean norm** $\|\mathbf{v}\| = \sqrt{\langle \mathbf{v}, \mathbf{v} \rangle}$ is its length. Two vectors are *orthogonal* when $\langle \mathbf{u}, \mathbf{v} \rangle = 0$ and *parallel* when $|\cos\theta| = 1$.

1.3 Geometry of vectors

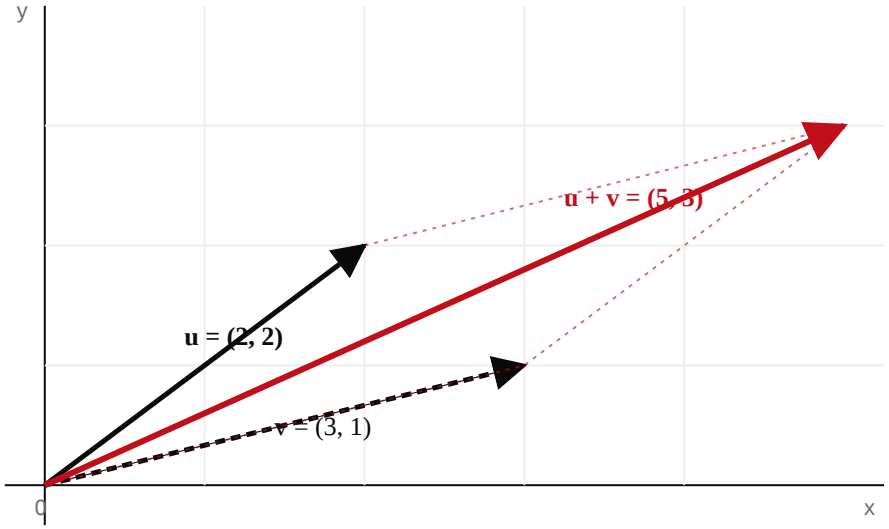


Figure 1.1 Vector addition as the diagonal of a parallelogram.

1.4 Worked example

Let $\mathbf{u} = (2, 2)^\top$ and $\mathbf{v} = (3, 1)^\top$.

- $\mathbf{u} + \mathbf{v} = (5, 3)^\top$.
- $\langle \mathbf{u}, \mathbf{v} \rangle = 2 \cdot 3 + 2 \cdot 1 = 8$.
- $\|\mathbf{u}\| = \sqrt{8} \approx 2.83$, $\|\mathbf{v}\| = \sqrt{10} \approx 3.16$.
- $\cos\theta = 8/(\sqrt{8}\sqrt{10}) \approx 0.894 \Rightarrow \theta \approx 26.6^\circ$.

DATA SCIENTIST LENS

Cosine similarity in NLP, recommender systems, and image retrieval is literally $\cos\theta$ from equation (1.1). Every sentence-embedding API you have ever called computes vector dot products and norms under the hood.

```
import numpy as np
u = np.array([2, 2]); v = np.array([3, 1])
print(u + v)                    # [5 3]
print(u @ v)                    # 8   (dot product)
print(np.linalg.norm(u))        # 2.828...
cos = (u @ v) / (np.linalg.norm(u) * np.linalg.norm(v))
print(np.degrees(np.arccos(cos)))  # 26.56...
```

MCQ 1

Q. Two vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^3$ satisfy $\|\mathbf{u}\| = 3,\ \|\mathbf{v}\| = 4$ and $\mathbf{u} \cdot \mathbf{v} = 0$. What is $\|\mathbf{u} + \mathbf{v}\|$?

A. 1

B. 5

C. 7

D. $\sqrt{7}$

Answer: B. Orthogonality gives $\|\mathbf{u} + \mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 = 9 + 16 = 25$ (Pythagoras in $\mathbb{R}^n$).

PROBLEM 1.1

Given $\mathbf{a} = (1, -2, 2)^\top$ and $\mathbf{b} = (3, 0, -4)^\top$, compute (i) $\mathbf{a} + \mathbf{b}$, (ii) $\mathbf{a} \cdot \mathbf{b}$, (iii) the angle θ between them, and (iv) a unit vector in the direction of $\mathbf{a}$.

PROBLEM 1.2

A 3-feature data point is $\mathbf{x} = (0.5, -1.2, 3.0)^\top$ and a learned weight vector is $\mathbf{w} = (2, -1, 0.5)^\top$. Compute the model's linear score $\mathbf{w}^\top \mathbf{x} + b$ for bias $b = -0.25$. State the predicted class if the decision rule is $\hat{y} = 1$ when the score > 0.

RECAP CARD

- Vectors add component-wise; scalars stretch them.
- Dot product $= \|\mathbf{u}\|\|\mathbf{v}\| \cos\theta$; orthogonal iff zero.
- Norm $= \sqrt{\mathbf{v}^\top \mathbf{v}}$; unit vector $\hat{\mathbf{v}} = \mathbf{v}/\|\mathbf{v}\|$.
- Cosine similarity, linear model scores, and gradient steps are all dot products.

02 CHAPTER TWO · TABLES OF NUMBERS THAT COMPUTE

Matrices & Matrix Operations

A **matrix** is a rectangular grid of numbers. For the data scientist, a dataset of m samples with n features is an $m \times n$ matrix X. Matrices are not just storage — they are *functions*: multiplying by a matrix transforms one vector space into another.

2.1 Matrix algebra

DEFINITION · $M \times N$ MATRIX

$A \in \mathbb{R}^{m \times n}$ is a grid with entries a_{ij} ($i \in \{1..m\}$ rows, $j \in \{1..n\}$ columns). Addition and scaling are component-wise. The product AB is defined when the inner dimensions match (A is $m \times k$, B is $k \times n$) and yields an $m \times n$ matrix with $(AB)_{ij} = \sum_{\ell=1}^{k} a_{i\ell} b_{\ell j}$.

Matrix multiplication is **associative** ($A(BC) = (AB)C$), **distributive** over addition, but **not commutative**: $AB \neq BA$ in general.

2.2 Matrix–vector multiplication, two views

$A\mathbf{x}$ can be read two ways:

1. **Row view.** The i-th entry of $A\mathbf{x}$ is the dot product of row i of A with $\mathbf{x}$.
2. **Column view.** $A\mathbf{x} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \cdots + x_n\mathbf{a}_n$ — a linear combination of the columns of A. (This is the single most useful picture in linear algebra.)

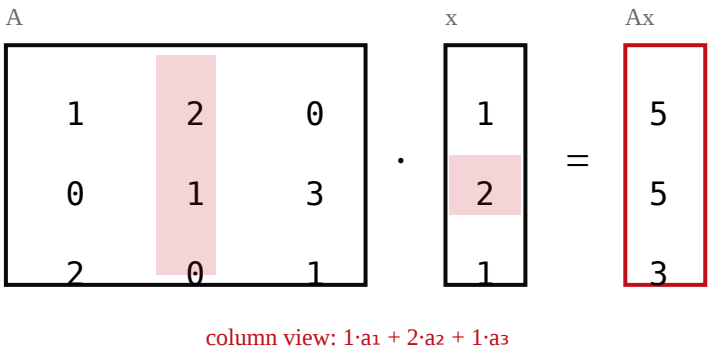


Figure 2.1 $A\mathbf{x}$ as a linear combination of columns of A.

2.3 Special matrices

Matrix	Definition	Why a data scientist cares
Identity I_n	$I_{ij} = 1$ if $i = j$, else 0	Initialisation; "no-op"

Matrix	Definition	Why a data scientist cares
Diagonal D	Only $d_{ii} \neq 0$	Feature scaling, SVD singular values
Symmetric $A = A^\top$	Mirror across the diagonal	Covariance, kernels, Hessians
Orthogonal Q	$Q^\top Q = I$	Rotations; numerical stability
Permutation P	Single 1 per row/col	Re-ordering rows/columns

2.4 Transpose and inverse (preview)

$(A^\top)_{ij} = a_{ji}$. Useful identities: $(AB)^\top = B^\top A^\top$ and, when the inverse exists, $(AB)^{-1} = B^{-1}A^{-1}$. The inverse satisfies $A^{-1}A = AA^{-1} = I$ and is only defined for square, full-rank matrices — a subject we return to in Chapter 4.

DATA SCIENTIST LENS

In a neural network's dense layer `y = Wx + b`, W is a matrix, $\mathbf{x}$ a batch of activations, and *training* means nudging the entries of W in the direction of the negative gradient. Understanding how W re-combines inputs is understanding what the layer *learns*.

```
import numpy as np
A = np.array([[1, 2, 0],
              [0, 1, 3],
              [2, 0, 1]])
x = np.array([1, 2, 1])
print(A @ x)         # [5 5 3]
print(A.T)           # transpose
print(np.linalg.matrix_rank(A))  # 3 (full rank)
```

MCQ 2

Q. If $A \in \mathbb{R}^{3\times 4}$ and $B \in \mathbb{R}^{4\times 2}$, which of the following is defined?

A. AB is 3×2

B. BA is 2×3

C. $A + B$

D. $AB^\top$ is 3×4

Answer: A. Inner dimensions (4 and 4) match; the result inherits outer dimensions 3 and 2. BA is not defined because $2 \neq 3$. $A + B$ is not defined because shapes differ.

PROBLEM 2.1

Let $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and $B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$. Compute AB, BA, and verify that $(AB)^\top = B^\top A^\top$.

PROBLEM 2.2

A dataset $X \in \mathbb{R}^{1000\times 50}$ is mean-centred and multiplied by a weight matrix $W \in \mathbb{R}^{50\times 10}$ to produce logits $Z = XW$. What is the shape of Z? How many scalar multiplications does computing Z take?

RECAP CARD

- Matrix $\times$ vector = linear combination of columns.
- Inner dimensions must match for multiplication.
- Not commutative, but associative and distributive.
- Special shapes (diagonal, symmetric, orthogonal) appear everywhere in ML.

03 CHAPTER THREE · SOLVING $A\mathbf{X} = \mathbf{B}$

Systems of Linear Equations

A linear system is a set of m equations in n unknowns. In matrix form it is $A\mathbf{x} = \mathbf{b}$. Three outcomes are possible: one unique solution, infinitely many, or none. Which case you are in depends entirely on how the columns of A span the right-hand side $\mathbf{b}$.

3.1 Row-echelon and reduced row-echelon form

DEFINITION · ROW ECHELON FORM (REF)

A matrix is in *row echelon form* if: (i) all-zero rows are at the bottom, (ii) the leading (first non-zero) entry of each non-zero row — called a **pivot** — is to the right of the pivot in the row above. If, in addition, every pivot is 1 and is the only non-zero entry in its column, the matrix is in *reduced row echelon form (RREF)*.

3.2 Gaussian elimination

Three *elementary row operations* preserve the solution set: (1) swap two rows, (2) multiply a row by a non-zero scalar, (3) add a multiple of one row to another. Apply them to the augmented matrix $[A \mid \mathbf{b}]$ until REF/RREF is reached, then back-substitute.

```
⌈                                                ⌉
| 1 2 1 | 4 |                                    | 1 2 1 | 4 |
| 2 5 3 | 9 |        R2 ← R2 − 2R1               | 0 1 1 | 1 |
| -1 1 2 | 5 |  ⟶    R3 ← R3 + R1                | 0 3 3 | 9 |
⌊                                                ⌋
                     (make column 1 zero below pivot)
```

Figure 3.1 One elimination step: zero out the entries below the first pivot.

3.3 Worked example

Solve

$$\begin{aligned} x + 2y + z &= 4 \\ 2x + 5y + 3z &= 9 \\ -x + y + 2z &= 5. \end{aligned}$$

After $R_2 \leftarrow R_2 - 2R_1$ and $R_3 \leftarrow R_3 + R_1$:

$$\begin{pmatrix} 1 & 2 & 1 \\ 0 & 1 & 1 \\ 0 & 3 & 3 \end{pmatrix} \mathbf{x} = \begin{pmatrix} 4 \\ 1 \\ 9 \end{pmatrix}.$$

Then $R_3 \leftarrow R_3 - 3R_2$ gives a zero row with right-hand side 6 — a contradiction, so the system is **inconsistent**. (If the RHS were 3 instead we would have infinitely many solutions.)

3.4 Existence and uniqueness

Situation	Condition	Picture
Unique solution	$\text{rank}(A) = \text{rank}(A\|\mathbf{b}) = n$	Lines intersect at a single point
Infinitely many	$\text{rank}(A) = \text{rank}(A\|\mathbf{b}) < n$	Lines coincide / planes share a line
No solution	$\text{rank}(A) < \text{rank}(A\|\mathbf{b})$	Parallel lines / skew planes

DATA SCIENTIST LENS

When a dataset has more features than samples ($n > m$), the system $X\boldsymbol{\beta} = \mathbf{y}$ is typically underdetermined — infinitely many $\boldsymbol{\beta}$ fit. Regularisation (Ridge, Lasso) picks one specific solution by adding a constraint.

```
import numpy as np
A = np.array([[1, 2, 1], [2, 5, 3], [-1, 1, 2]], float)
b = np.array([4, 9, 5], float)
# least-squares always returns *something*; check residual:
x, residuals, rank, sv = np.linalg.lstsq(A, b, rcond=None)
print(rank)        # < 3 would indicate deficiency
print(np.linalg.matrix_rank(np.c_[A, b]))  # compare augmented rank
```

MCQ 3

Q. A system $A\mathbf{x} = \mathbf{b}$ has $\text{rank}(A) = 2$ and $\text{rank}([A|\mathbf{b}]) = 3$. Which is true?

A. Unique solution

B. Infinitely many solutions

C. No solution

D. Cannot determine without A

Answer: C. The augmented matrix has higher rank than A, so $\mathbf{b}$ is not in the column space of A — the system is inconsistent.

PROBLEM 3.1

Solve by Gaussian elimination: $x + y - z = 2, \quad 2x - y + z = 3, \quad x - 2y + 2z = 1.$

PROBLEM 3.2

For which values of k does the system $x + y = 1, \quad 2x + ky = 2$ have (i) a unique solution, (ii) infinitely many, (iii) none?

RECAP CARD

- Elementary row operations never change the solution set.
- Pivot positions tell you rank.
- Rank of A vs rank of $[A|\mathbf{b}]$ determines existence / uniqueness.
- Underdetermined systems are the norm in high-dimensional ML.

04 CHAPTER FOUR · THE NUMBER THAT REVEALS SHAPE

Determinants & Invertibility

The **determinant** $\det(A)$ of a square matrix is a single number that encodes how A scales (signed) volume. When $\det(A) = 0$, the matrix squashes space onto a lower dimension; it cannot be inverted. When $|\det(A)|$ is small but non-zero, inversion is still possible but numerically dangerous.

4.1 2×2 and 3×3 determinants

For $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, $\det(A) = ad - bc$. For 3×3, use the rule of Sarrus or cofactor expansion along any row/column:

$$\det(A) = \sum_{j=1}^{n} (-1)^{i+j} a_{ij} M_{ij}, \tag{4.1}$$

where M_{ij} is the minor obtained by deleting row i and column j.

4.2 Properties

- $\det(I) = 1, \det(A^\top) = \det(A), \det(AB) = \det(A)\det(B)$.
- Swapping two rows changes the sign; scaling a row by α scales $\det$ by α; adding a multiple of one row to another leaves $\det$ unchanged.
- For triangular matrices, $\det$ is the product of diagonal entries.

4.3 Geometric meaning

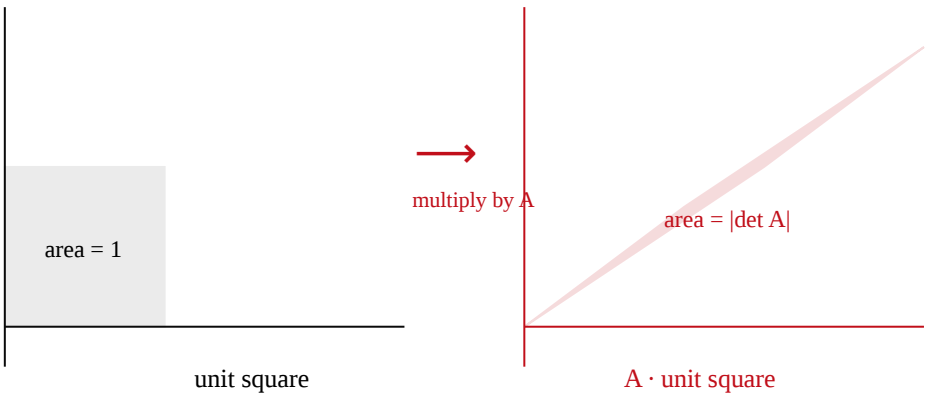


Figure 4.1 $|\det A|$ is the factor by which A stretches area (volume in higher dimensions).

4.4 The Invertibility Theorem

THEOREM · EQUIVALENT CONDITIONS FOR INVERTIBILITY

For a square matrix $A \in \mathbb{R}^{n \times n}$, the following are all equivalent:

1. A is invertible (i.e., A^{-1} exists).
2. $\det(A) \neq 0$.
3. $\text{rank}(A) = n$ (columns are linearly independent).
4. $A\mathbf{x} = \mathbf{0}$ only when $\mathbf{x} = \mathbf{0}$.
5. $A\mathbf{x} = \mathbf{b}$ has a unique solution for every $\mathbf{b}$.
6. 0 is not an eigenvalue of A.

If A^{-1} exists, Cramer's formula gives $\mathbf{x} = A^{-1}\mathbf{b}$, but in practice you should almost never compute A^{-1} directly — solve the linear system instead (Chapter 10).

DATA SCIENTIST LENS

A covariance matrix Σ is singular when features are linearly dependent — a classic cause of the "singular matrix" error when fitting linear regression without regularisation. Ridge regression adds λI to guarantee invertibility: $(X^\top X + \lambda I)^{-1}$.

```
A = np.array([[1, 2], [3, 4]], float)
print(np.linalg.det(A))        # -2.0
print(np.linalg.inv(A))        # [[-2,  1], [ 1.5, -0.5]]
# Prefer solve() over inv() for Ax=b:
b = np.array([5, 11])
print(np.linalg.solve(A, b))   # [1, 2]
```

MCQ 4

Q. If $A \in \mathbb{R}^{n \times n}$ and $\det(A) = 0$, which statement must be true?

A. A has at least one zero row.

B. The columns of A are linearly dependent.

C. A is the zero matrix.

D. $A = A^\top$.

Answer: B. A zero determinant is equivalent to rank-deficiency, which is equivalent to the columns being linearly dependent. (A) and (C) are too strong; (D) is unrelated.

PROBLEM 4.1

Compute $\det \begin{pmatrix} 2 & 1 & 3 \\ 0 & -1 & 4 \\ 1 & 2 & 0 \end{pmatrix}$ by cofactor expansion along the first column.

PROBLEM 4.2

Find all values of λ for which the matrix $A - \lambda I$ is singular, where $A = \begin{pmatrix} 3 & 1 \\ 0 & 2 \end{pmatrix}$. (Preview of eigenvalues!)

RECAP CARD

- $\det(A) = 0 \Leftrightarrow A$ is not invertible $\Leftrightarrow$ columns dependent.
- Determinants multiply: $\det(AB) = \det(A)\det(B)$.
- Geometrically $|\det A|$ is a volume scale factor.
- Never compute A^{-1} in practice — solve the system instead.

05 CHAPTER FIVE · STRUCTURE OF SOLUTIONS

Basis, Dimension & Rank

Every matrix defines four natural subspaces. Understanding them explains *why* a regression model has a unique solution, *when* a least-squares fit exists, and *where* redundant features hide.

5.1 Linear independence and span

DEFINITION · LINEAR INDEPENDENCE

Vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$ are *linearly independent* if $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0}$ implies $c_1 = \cdots = c_k = 0$. Their *span* is the set of all linear combinations — a subspace of $\mathbb{R}^n$.

5.2 Basis and dimension

A **basis** of a subspace is an independent spanning set. Every vector in the subspace has a unique expansion in that basis. The number of basis vectors is the **dimension** — an intrinsic property that does not depend on which basis you pick.

5.3 The four fundamental subspaces

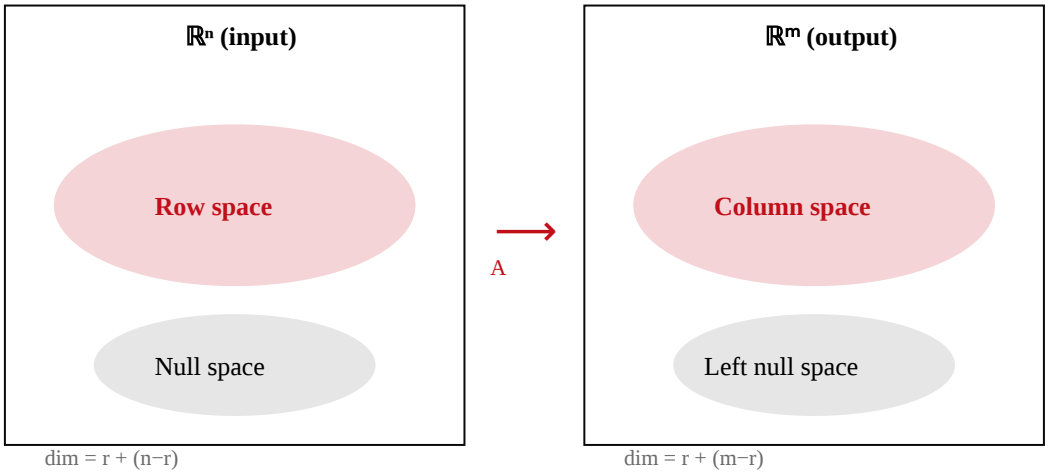


Figure 5.1 The four fundamental subspaces, with $r = \text{rank}(A)$.

Subspace	Definition	Lives in	Dimension
Column space $C(A)$	span of columns	$\mathbb{R}^m$	r
Null space $N(A)$	$\{\mathbf{x} : A\mathbf{x} = \mathbf{0}\}$	$\mathbb{R}^n$	$n - r$
Row space $C(A^\top)$	span of rows	$\mathbb{R}^n$	r
Left null space $N(A^\top)$	$\{\mathbf{y} : A^\top\mathbf{y} = \mathbf{0}\}$	$\mathbb{R}^m$	$m - r$

THEOREM · RANK–NULLITY

For $A \in \mathbb{R}^{m\times n}$, $\operatorname{rank}(A) + \dim N(A) = n$.

5.4 Worked example

Let $A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{pmatrix}$. Row reducing gives a single pivot, so rank$= 1$. The null space is $\{(x, y, z) : x + 2y + 3z = 0\}$, two-dimensional (a plane). Check: $1 + 2 = 3 = n$.

DATA SCIENTIST LENS

In a design matrix X, perfectly collinear features (e.g., temperature in °C and °F) live in the row space and make rank$(X) < n$. Their coefficient vector lies in the null space — no unique least-squares solution. Dropping one of the collinear features is the cleanest fix.

```
A = np.array([[1,2,3],[2,4,6]], float)
print(np.linalg.matrix_rank(A))      # 1
# Null space via SVD:
U, S, Vt = np.linalg.svd(A)
ns = Vt[A.shape[0]:].T                # columns span N(A)
print(ns.shape, "  A @ ns ≈", np.allclose(A @ ns, 0))
```

MCQ 5

Q. $A \in \mathbb{R}^{5\times 7}$ has rank 4. What is $\dim N(A)$?

A. 1

B. 3

C. 4

D. 7

Answer: B. By rank–nullity, $\dim N(A) = n - r = 7 - 4 = 3$.

PROBLEM 5.1

Find a basis for the column space and null space of $A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{pmatrix}$.

PROBLEM 5.2

Show that if $\mathbf{v} \in N(A)$ and $\mathbf{u} \in C(A^\top)$, then $\mathbf{u}^\top \mathbf{v} = 0$.

RECAP CARD

- Rank = pivots = dimension of row / column space.
- Rank–nullity: columns minus pivots $=$ nullity.
- Null space $\perp$ row space; left null $\perp$ column space.
- Linear dependence among features = non-trivial null space.

06 CHAPTER SIX · MATRICES AS FUNCTIONS

Linear Transformations

A **linear transformation** $T : \mathbb{R}^n \to \mathbb{R}^m$ satisfies $T(\alpha\mathbf{u} + \beta\mathbf{v}) = \alpha T(\mathbf{u}) + \beta T(\mathbf{v})$. Once you fix bases, every linear transformation is captured by a unique matrix.

6.1 The matrix of a transformation

If $T(\mathbf{e}_j) = \mathbf{a}_j$ for the j-th standard basis vector, then $T(\mathbf{x}) = A\mathbf{x}$ with $A = [\mathbf{a}_1 \mid \mathbf{a}_2 \mid \ldots \mid \mathbf{a}_n]$. So: *apply T to each basis vector and stack the results as columns.*

6.2 Gallery of 2-D linear maps

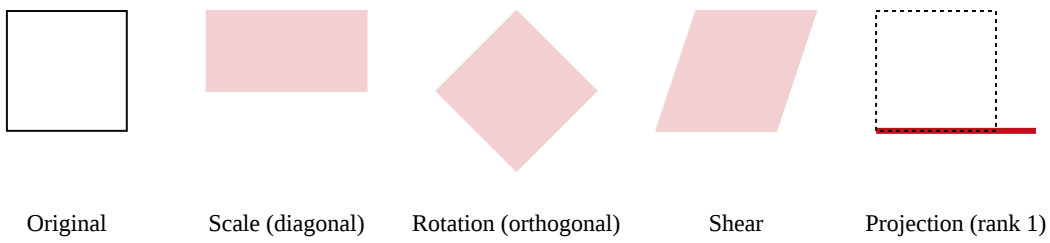


Figure 6.1 Linear transformations preserve straight lines and the origin.

Common 2-D matrices:

$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}, \quad S_{a,b} = \begin{pmatrix} a & 0 \\ 0 & b \end{pmatrix}, \quad H_k = \begin{pmatrix} 1 & k \\ 0 & 1 \end{pmatrix}.$$

6.3 Composition and change of basis

If $T = T_1 \circ T_2$ then the matrix of T is the product of matrices in the *same order*: $A = A_1 A_2$. Given a new basis B, the matrix of T in that basis is $[T]_B = P^{-1}AP$ where P is the change-of-basis matrix with B's vectors as columns. We will see $P^{-1}AP$ again in Chapter 8 — it is the essence of diagonalisation.

6.4 Kernel and image

$\ker(T) = \{\mathbf{x} : T(\mathbf{x}) = \mathbf{0}\} = N(A)$ and $\operatorname{im}(T) = C(A)$. T is *injective* iff $\ker(T) = \{\mathbf{0}\}$, *surjective* iff $\operatorname{im}(T) = \mathbb{R}^m$, and *invertible* iff both, which forces $m = n$.

DATA SCIENTIST LENS

A dense layer W is a linear transformation; a non-linear activation σ is what lets a neural network escape pure linearity. Without σ, a 100-layer MLP would collapse into a single matrix $W = W_{100} \cdots W_1$ — no more expressive than a one-layer linear model.

```
theta = np.deg2rad(30)
R = np.array([[np.cos(theta), -np.sin(theta)],
              [np.sin(theta),  np.cos(theta)]])
S = np.diag([2.0, 0.5])
T = R @ S                   # first scale, then rotate
print(T @ np.array([1, 0])) # image of e1
```

MCQ 6

Q. Which of these functions is *not* a linear transformation?

A. $T(\mathbf{x}) = A\mathbf{x}$ with $A \in \mathbb{R}^{m\times n}$
B. $T(\mathbf{x}) = 2\mathbf{x}$
C. $T(\mathbf{x}) = A\mathbf{x} + \mathbf{b}$ with fixed $\mathbf{b} \neq \mathbf{0}$
D. $T(\mathbf{x}) = 0$

Answer: C. Affine maps fail the zero-preservation test: $T(\mathbf{0}) = \mathbf{b} \neq \mathbf{0}$. (A), (B), (D) are all linear.

PROBLEM 6.1

Write the 2×2 matrix of the transformation that first rotates by 90° counter-clockwise and then reflects across the y-axis. Apply it to $(1, 0)^\top$.

PROBLEM 6.2

Let $T : \mathbb{R}^3 \to \mathbb{R}^2$ be $T(x, y, z) = (x + y,\ y - z)$. Write its matrix and describe $\ker(T)$ and $\operatorname{im}(T)$
.

RECAP CARD

- Linear $\Leftrightarrow$ respects addition and scaling; fixes origin.
- The matrix of T has $T(\mathbf{e}_j)$ as its j-th column.
- Composition $\leftrightarrow$ matrix product (order matters).
- Kernel = null space, image = column space.

07 CHAPTER SEVEN · FROM GEOMETRY TO REGRESSION

Orthogonality, Projections & Least Squares

Linear regression is the single most-used model in data science, and it is *projection* in disguise. When $A\boldsymbol{\beta} = \mathbf{y}$ has no exact solution, we instead find the $\boldsymbol{\beta}$ whose prediction $A\boldsymbol{\beta}$ is the closest point to $\mathbf{y}$ inside the column space of A — the orthogonal projection.

7.1 Orthogonal sets & Gram–Schmidt

A set $\{\mathbf{q}_1, \ldots, \mathbf{q}_k\}$ is *orthonormal* if $\langle \mathbf{q}_i, \mathbf{q}_j \rangle = \delta_{ij}$. Any independent set can be **orthonormalised**:

$$\mathbf{u}_k = \mathbf{v}_k - \sum_{j<k} \langle \mathbf{v}_k, \mathbf{q}_j \rangle \mathbf{q}_j, \qquad \mathbf{q}_k = \mathbf{u}_k / \|\mathbf{u}_k\|.$$

7.2 Orthogonal projection

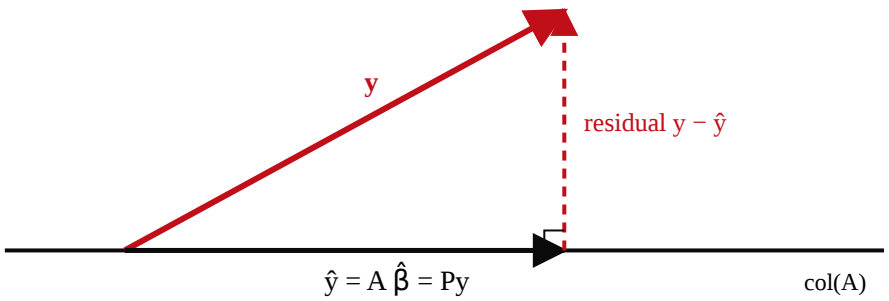


Figure 7.1 Orthogonal projection — the residual is perpendicular to the column space.

If the columns of A are independent, the projection of $\mathbf{y}$ onto $C(A)$ is

$$P = A(A^\top A)^{-1} A^\top, \qquad \hat{\mathbf{y}} = P\mathbf{y}.$$

P satisfies $P = P^\top$ and $P^2 = P$ — it is idempotent and symmetric. (7.1)

7.3 The normal equations (linear regression)

THEOREM · LEAST SQUARES

The $\boldsymbol{\beta}$ that minimises $\|\mathbf{y} - A\boldsymbol{\beta}\|^2$ satisfies the *normal equations*

$$A^\top A\, \boldsymbol{\beta} = A^\top \mathbf{y}.$$

If A has full column rank, the unique solution is $\boldsymbol{\beta} = (A^\top A)^{-1} A^\top \mathbf{y}$.

7.4 Worked example

Fit $y = \beta_0 + \beta_1 x$ to the points $(1,1), (2,2), (3,2)$.

$A = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}$, $\mathbf{y} = \begin{pmatrix} 1 \\ 2 \\ 2 \end{pmatrix}$. Then $A^\top A = \begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix}$, $A^\top \mathbf{y} = \begin{pmatrix} 5 \\ 11 \end{pmatrix}$. Solving gives $\hat{\boldsymbol{\beta}} = (2/3,\ 1/2)^\top$, so $\hat{y} = 2/3 + x/2$.

DATA SCIENTIST LENS

Scikit-learn's `LinearRegression().fit()` solves exactly equation (7.1) under the hood (usually via QR or SVD, not by literally inverting $A^\top A$ — see Chapters 10–11). Ridge: $(A^\top A + \lambda I)\boldsymbol{\beta} = A^\top \mathbf{y}$.

```
X = np.array([[1,1],[1,2],[1,3]], float)
y = np.array([1,2,2], float)
beta, *_ = np.linalg.lstsq(X, y, rcond=None)
print(beta)                # [0.666..., 0.5]
y_hat = X @ beta
residual = y - y_hat
print(X.T @ residual)      # ≈ 0 (orthogonality)
```

MCQ 7

Q. Which property is *not* true of a projection matrix $P = A(A^\top A)^{-1}A^\top$?

A. $P^2 = P$

B. $P^\top = P$

C. P is invertible

D. $P\mathbf{y}$ is the closest point in $C(A)$ to $\mathbf{y}$

Answer: C. A non-trivial projection has rank $< n$ (it squashes the component orthogonal to $C(A)$ to zero), so it is not invertible.

PROBLEM 7.1

Given $A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}$ and $\mathbf{y} = (6,0,0)^\top$, find the least-squares $\boldsymbol{\beta}$ and verify that the residual is orthogonal to every column of A.

PROBLEM 7.2

Apply Gram–Schmidt to $\mathbf{v}_1 = (1,1,0)^\top$, $\mathbf{v}_2 = (1,0,1)^\top$ to obtain an orthonormal basis for their span.

RECAP CARD

- Projection = closest point in a subspace; residual $\perp$ subspace.
- Normal equations $A^\top A\boldsymbol{\beta} = A^\top \mathbf{y}$ = OLS regression.
- Gram–Schmidt builds orthonormal bases.
- Ridge adds λI to stabilise $A^\top A$.

08 CHAPTER EIGHT · HIDDEN AXES OF A MATRIX

Eigenvalues & Eigenvectors

Some vectors are stretched but *not rotated* by a matrix; they are its **eigenvectors**, and the stretch factors are the **eigenvalues**. These hidden axes diagonalise the matrix and reveal its long-run behaviour.

DEFINITION · EIGENVALUE / EIGENVECTOR

A non-zero $\mathbf{v} \in \mathbb{R}^n$ is an eigenvector of A with eigenvalue λ if $A\mathbf{v} = \lambda\mathbf{v}$. Equivalently, $(A - \lambda I)\mathbf{v} = \mathbf{0}$, so $\det(A - \lambda I) = 0$.

8.1 The characteristic polynomial

$p_A(\lambda) = \det(A - \lambda I)$ is a polynomial of degree n. Its roots are the eigenvalues. Eigenvectors are found by solving $(A - \lambda I)\mathbf{v} = \mathbf{0}$.

8.2 Diagonalisation

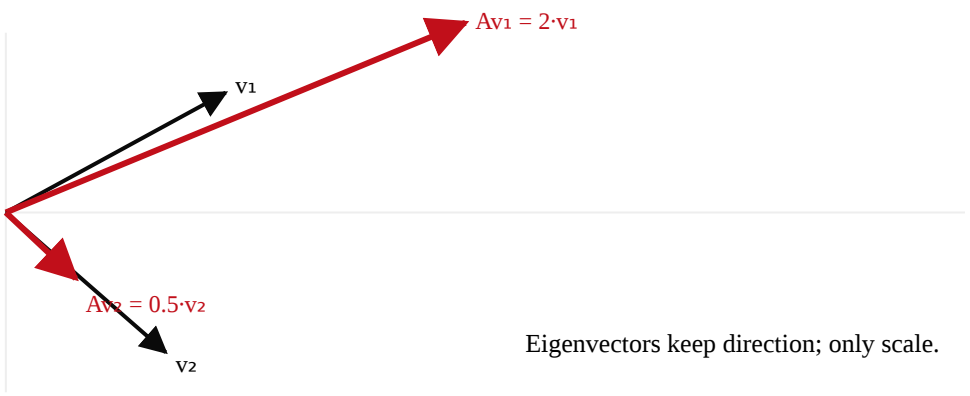


Figure 8.1 Eigenvectors of A are directions in which A acts as pure scaling.

If A has n independent eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_n$, stacking them as columns of V gives

$$A = V\Lambda V^{-1}, \quad \Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_n).$$

Then $A^k = V\Lambda^k V^{-1}$, which is why eigendecomposition diagonalises powers of matrices (Markov chains, PageRank, recurrences).

8.3 Symmetric matrices — the best case

SPECTRAL THEOREM

Every real symmetric matrix $A = A^\top$ has real eigenvalues and an orthonormal basis of eigenvectors. It can be written $A = Q\Lambda Q^\top$ with Q orthogonal.

This is why covariance matrices, kernel matrices, and Hessians — all symmetric — are so well-behaved.

8.4 Worked example

$A = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$. $\det(A - \lambda I) = (4-\lambda)(3-\lambda) - 2 = \lambda^2 - 7\lambda + 10 = (\lambda - 5)(\lambda - 2)$.
Eigenvalues $5, 2$. For $\lambda = 5$: $(A - 5I)\mathbf{v} = \mathbf{0} \Rightarrow \mathbf{v} = (1,1)^\top$. For $\lambda = 2$: $\mathbf{v} = (1,-2)^\top$.

DATA SCIENTIST LENS

The power method — repeatedly applying A to a random vector and normalising — converges to the top eigenvector. That is exactly how Google's original PageRank was computed on the web's hyperlink matrix.

```
A = np.array([[4,1],[2,3]], float)
vals, vecs = np.linalg.eig(A)
print(vals)                 # [5. 2.]
# Verify A v = λ v:
for λ, v in zip(vals, vecs.T):
    print(np.allclose(A @ v, λ * v))
```

MCQ 8

Q. If A is $n \times n$ and λ is an eigenvalue of A, which is always an eigenvalue of A^2?

A. $\lambda + 1$
B. λ^2
C. 2λ
D. $1/\lambda$

Answer: B. If $A\mathbf{v} = \lambda\mathbf{v}$ then $A^2\mathbf{v} = A(\lambda\mathbf{v}) = \lambda A\mathbf{v} = \lambda^2\mathbf{v}$.

PROBLEM 8.1

Find all eigenvalues and a basis of eigenvectors for $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. Verify the spectral theorem by checking orthogonality.

PROBLEM 8.2

Let $A = \begin{pmatrix} 0.9 & 0.2 \\ 0.1 & 0.8 \end{pmatrix}$ be a Markov transition matrix. Show that $\lambda = 1$ is an eigenvalue and find its eigenvector — this is the stationary distribution.

RECAP CARD

- Eigenvectors = directions left invariant by A.
- Diagonalisation $A = V\Lambda V^{-1}$ makes powers easy.
- Symmetric $\Rightarrow$ real eigenvalues + orthonormal eigenvectors.
- Power iteration $\Rightarrow$ dominant eigenvector (PageRank, PCA).

09 CHAPTER NINE · THE SWISS-ARMY KNIFE

SVD & Principal Component Analysis

The **singular value decomposition** works for *every* matrix — rectangular, rank-deficient, noisy. It is the most important factorisation in applied mathematics and the mathematical backbone of PCA, latent-semantic analysis, collaborative filtering, and modern matrix completion.

THEOREM · SVD

Every $A \in \mathbb{R}^{m\times n}$ can be written

$$A = U\Sigma V^\top,$$

with $U \in \mathbb{R}^{m\times m}$ orthogonal, $V \in \mathbb{R}^{n\times n}$ orthogonal, and $\Sigma \in \mathbb{R}^{m\times n}$ diagonal with non-negative singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$.

9.1 Intuition

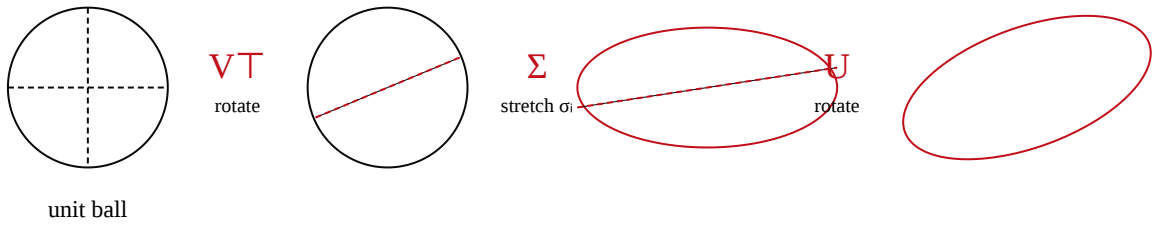


Figure 9.1 Any linear map = rotate, stretch along axes, rotate. Singular values are the stretches.

9.2 Low-rank approximation (Eckart–Young)

Truncating the SVD to the top k singular values,

$$A_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top,$$

gives the best rank-k approximation of A in the Frobenius norm. This is how *every* matrix-compression scheme (JPEG-style image compression, model distillation for dense layers, topic modelling) ultimately works.

9.3 Principal Component Analysis (PCA)

Given a centred data matrix $X \in \mathbb{R}^{m\times n}$:

1. Compute SVD $X = U\Sigma V^\top$.

2. The columns of V are the **principal directions** (orthonormal features of maximum variance).
3. The *scores* $XV = U\Sigma$ are the data rotated into the PC coordinate system.
4. Variance explained by PC$_i$ is $\sigma_i^2/(m-1)$.

Equivalently, PCA diagonalises the covariance matrix $\Sigma_X = \frac{1}{m-1}X^\top X = V\Lambda V^\top$ with $\lambda_i = \sigma_i^2/(m-1)$.

DATA SCIENTIST LENS

Before training a model with thousands of correlated features, run PCA and keep enough components to explain ~95% variance. You usually get the same model accuracy with 10–20× fewer features and dramatically faster training.

```
X = np.random.randn(200, 50)
X -= X.mean(0)
U, S, Vt = np.linalg.svd(X, full_matrices=False)
explained = (S**2) / (S**2).sum()
k = np.searchsorted(np.cumsum(explained), 0.95) + 1
scores = U[:, :k] * S[:k]                    # X @ V[:, :k]
reconstruction = scores @ Vt[:k]             # ≈ X for k large
print(k, "components retain 95% variance")
```

MCQ 9

Q. The best rank-k approximation of A in the Frobenius norm is obtained by:

A. Keeping the k largest entries of A
B. Zeroing the k smallest eigenvalues of $A^\top A$
C. Keeping the k largest singular values of A
D. Projecting A onto a random k-dimensional subspace

Answer: C. This is the Eckart–Young–Mirsky theorem.

PROBLEM 9.1

Compute (by hand, to 2 decimal places) the SVD of $A = \begin{pmatrix} 3 & 0 \\ 4 & 5 \end{pmatrix}$ by first finding the eigenvalues of $A^\top A$

.

PROBLEM 9.2

An $m \times n$ dataset has singular values $\sigma = (10, 8, 2, 0.1, 0.05)$. Plot the scree (cumulative variance). How many components capture 95% of the variance?

RECAP CARD

- Every matrix: $A = U\Sigma V^\top$ — rotate, stretch, rotate.
- Top-k SVD = best rank-k approximation (Eckart–Young).
- PCA = SVD of centred data; variance = $\sigma_i^2/(m-1)$.
- Compression, denoising, LSA, recsys — all leverage truncated SVD.

10 CHAPTER TEN · PRACTICAL SOLVERS
Matrix Factorisations

In production you never solve $A\mathbf{x} = \mathbf{b}$ with pen-and-paper Gaussian elimination; you use a *factorisation*. Three dominate numerical practice: LU, QR, and Cholesky.

10.1 LU decomposition

Any (permuted) square A admits $PA = LU$ with L unit-lower-triangular and U upper-triangular. Once factored, solve two triangular systems: $L\mathbf{y} = P\mathbf{b}$, then $U\mathbf{x} = \mathbf{y}$. Great when you need to solve many systems with the same A.

10.2 QR decomposition

Every $A \in \mathbb{R}^{m\times n}$ with $m \geq n$ admits $A = QR$ with Q orthogonal columns and R upper-triangular. For least-squares, $A\boldsymbol{\beta} = \mathbf{y} \Rightarrow R\boldsymbol{\beta} = Q^\top \mathbf{y}$. More numerically stable than normal equations.

10.3 Cholesky decomposition

If A is symmetric positive-definite, $A = LL^\top$ with L lower-triangular. Twice as fast as LU and the workhorse of Gaussian processes, Kalman filters, and Bayesian inference.

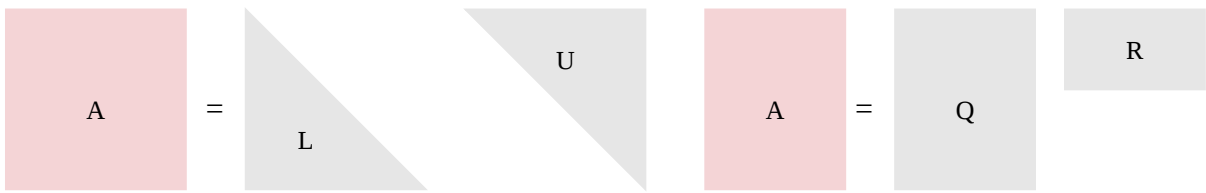


Figure 10.1 Shapes of the LU and QR factorisations.

10.4 When to use what

Problem	Best factorisation	Cost (dense, $n \times n$)
General $A\mathbf{x} = \mathbf{b}$	LU with partial pivoting	$\frac{2}{3}n^3$
Least squares, rectangular	QR (Householder)	$2mn^2 - \frac{2}{3}n^3$
Symmetric positive definite	Cholesky	$\frac{1}{3}n^3$
Rank-deficient / ill-conditioned	SVD (Ch. 9)	$O(mn^2)$

DATA SCIENTIST LENS

`scipy.linalg.cho_solve` vs `scipy.linalg.solve` can be a 2× speedup on Gaussian process regression where the kernel matrix is SPD. Always check whether your matrix has exploitable structure.

```
from scipy.linalg import lu, qr, cho_factor, cho_solve
A = np.array([[4, 12, -16],
              [12, 37, -43],
              [-16, -43, 98]], float)   # SPD
c, low = cho_factor(A)                  # Cholesky
x = cho_solve((c, low), np.array([1, 2, 3], float))
Q, R = qr(A)                             # QR
P, L, U = lu(A)                          # LU
```

MCQ 10

Q. Which factorisation is the *most numerically stable* choice for solving a rank-deficient least-squares problem?

A. LU

B. Cholesky

C. QR

D. SVD

Answer: D. SVD handles rank deficiency directly via the Moore–Penrose pseudoinverse $A^+ = V\Sigma^+U^\top$.

PROBLEM 10.1

By hand, compute the LU decomposition (no pivoting) of $A = \begin{pmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{pmatrix}$.

PROBLEM 10.2

A kernel matrix $K = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$ is SPD. Find its Cholesky factor L such that $K = LL^\top$.

RECAP CARD

- LU: general solve. QR: least squares. Cholesky: SPD. SVD: anything.
- Factor once, solve many times.
- Exploiting structure = order-of-magnitude speedups.

11 CHAPTER ELEVEN · WHEN MATH MEETS FLOATING POINT

Numerical Linear Algebra

On a computer, every number is rounded. Two mathematically identical algorithms can produce wildly different answers. This chapter explains the concepts every practitioner needs: *conditioning*, *stability*, and *sparsity*.

11.1 Condition number

DEFINITION · CONDITION NUMBER

$\kappa(A) = \sigma_{\max}(A)/\sigma_{\min}(A)$. If κ is large, tiny perturbations of $\mathbf{b}$ can cause large perturbations of $\mathbf{x} = A^{-1}\mathbf{b}$.

Rule of thumb: if $\kappa(A) \approx 10^k$, expect to lose $\approx k$ digits of accuracy. Ridge regression and Tikhonov regularisation are methods of reducing the effective condition number.

11.2 Numerical stability: backward error

An algorithm is *backward stable* if it returns the exact solution to a slightly perturbed problem. Householder QR is backward stable; the textbook Gram–Schmidt is *not* — in floating point its "orthogonal" vectors drift. Use *modified* Gram–Schmidt, or (better) Householder reflections.

11.3 Sparse matrices

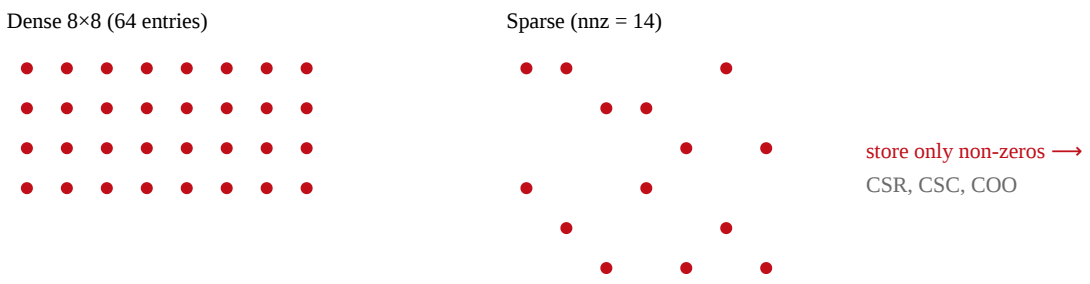


Figure 11.1 Typical TF–IDF / adjacency matrices are > 99% zero. Never store them densely.

11.4 Iterative solvers

For huge sparse systems (millions of unknowns), direct factorisation is infeasible. Iterative methods — *Conjugate Gradient* (SPD), *GMRES* (general), *Lanczos/Arnoldi* (top-k eigenvalues) — converge in far fewer iterations than the matrix has rows, using only matrix-vector products.

GOTCHA

Always check `np.linalg.cond(A)` before doing `np.linalg.solve(A, b)` on ill-conditioned problems. A condition number $> 10^{12}$ in double precision essentially returns noise.

```
from scipy.sparse import csr_matrix
from scipy.sparse.linalg import cg
A_sparse = csr_matrix(np.diag(np.arange(1, 1001, dtype=float)))
b = np.ones(1000)
x, info = cg(A_sparse, b, atol=1e-8)     # converges in O(√κ) steps
print(info)                               # 0 = converged
```

MCQ 11

Q. Doubling every entry of A changes the condition number $\kappa(A) = \sigma_{\max}/\sigma_{\min}$ to:

A. 2κ

B. $\kappa/2$

C. κ

D. 4κ

Answer: C. Scaling all singular values by 2 cancels in the ratio.

PROBLEM 11.1

Compute the condition number of $A = \begin{pmatrix} 1 & 1 \\ 1 & 1.0001 \end{pmatrix}$ (in NumPy if you like). Explain why tiny changes in $\mathbf{b}$ destabilise the solution.

PROBLEM 11.2

Why is Cholesky factorisation *twice as fast* as LU? Count the multiplications needed for an $n \times n$ SPD matrix.

RECAP CARD

- Condition number bounds the damage that round-off can do.
- Prefer backward-stable algorithms (Householder, modified G-S).
- Sparse storage is non-negotiable for real datasets.
- Iterative CG/GMRES scale where direct methods can't.

12 CHAPTER TWELVE · LINEAR ALGEBRA IN THE WILD
Applications in Data Science

This final chapter walks through four iconic data-science systems and pinpoints the linear algebra driving each.

12.1 Word embeddings & latent semantic analysis

Build the term–document matrix X, compute its truncated SVD $X_k = U_k \Sigma_k V_k^\top$. Rows of U_k give a k-dimensional vector for every word; $\cos$-similarity in that space captures meaning. The same idea powers item2vec, node2vec, and classical collaborative filtering.

12.2 PageRank

Let M be the column-stochastic adjacency matrix of the web. The *PageRank* vector $\mathbf{r}$ is the dominant eigenvector of the Google matrix

$$G = \alpha M + (1-\alpha)\tfrac{1}{n}\mathbf{1}\mathbf{1}^\top, \qquad \alpha \approx 0.85.$$

Power iteration converges in ~ 50 multiplications — cheap even for billions of pages because M is extremely sparse.

12.3 Recommender systems (matrix factorisation)

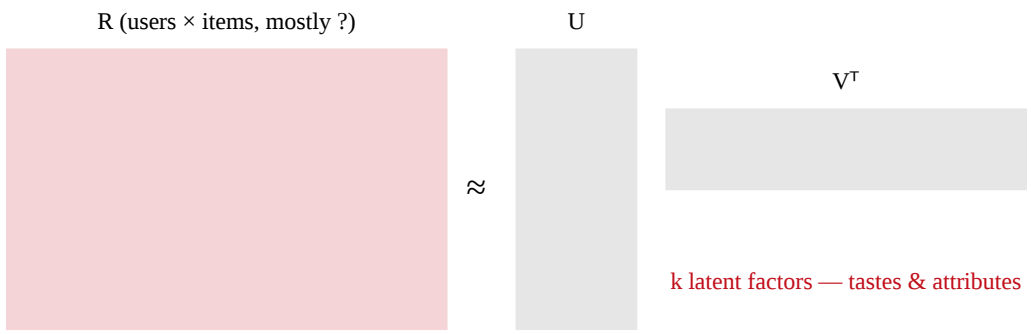


Figure 12.1 Netflix-style collaborative filtering = low-rank completion of a giant, mostly-missing matrix.

Fit latent user and item matrices U, V by minimising $\sum_{(i,j)\in\Omega}(R_{ij} - \mathbf{u}_i^\top \mathbf{v}_j)^2 + \lambda(\|\mathbf{u}_i\|^2 + \|\mathbf{v}_j\|^2)$ over the *observed* entries Ω (alternating least squares, SGD, or implicit ALS).

12.4 Neural networks

Every forward pass of a feed-forward network is a sequence of matrix multiplies interleaved with non-linearities:

$$\mathbf{h}_\ell = \sigma(W_\ell \mathbf{h}_{\ell-1} + \mathbf{b}_\ell).$$

Back-propagation is repeated application of the chain rule — which in matrix form is transpose, multiply, and element-wise Hadamard product with σ'. Attention in Transformers computes $\mathrm{softmax}(QK^\top/\sqrt{d})V$ — three

matrix products. Training on a GPU is essentially a festival of dense matrix multiplications.

DATA SCIENTIST LENS · TYING IT TOGETHER

- Regression → Chapter 7 (projections).
- Logistic regression gradient → Chapter 1 (dot products) + Chapter 2.
- PCA for viz → Chapter 9.
- Recommender systems → Chapters 9 + 10 (low-rank, ALS).
- Transformer attention → Chapters 2 + 8 (matrix products, eigenvalues for stability).
- Gaussian processes → Chapter 10 (Cholesky).

```
import numpy as np
# Toy matrix-factorisation recommender (ALS, 1 iteration)
R = np.array([[5,3,0,1],
              [4,0,0,1],
              [1,1,0,5],
              [0,0,4,4]], float)
mask = (R > 0).astype(float)
k = 2; rng = np.random.default_rng(0)
U = rng.standard_normal((4, k)); V = rng.standard_normal((4, k))
for _ in range(50):
    for i in range(4):
        obs = mask[i] == 1
        U[i] = np.linalg.lstsq(V[obs], R[i, obs], rcond=None)[0]
    for j in range(4):
        obs = mask[:, j] == 1
        V[j] = np.linalg.lstsq(U[obs], R[obs, j], rcond=None)[0]
print(np.round(U @ V.T, 2))     # filled-in ratings
```

MCQ 12

Q. In Transformer self-attention, the matrix $QK^\top$ has shape:

A. $(d \times d)$
B. $(n \times n)$ where n is sequence length
C. $(n \times d)$
D. $(1 \times n)$

Answer: B. $Q, K \in \mathbb{R}^{n \times d}$ so $QK^\top \in \mathbb{R}^{n \times n}$: a pairwise token-to-token score matrix.

PROBLEM 12.1

Given a 5×5 web adjacency matrix, compute one step of power iteration starting from $\mathbf{r}_0 = \frac{1}{5}\mathbf{1}$ with damping $\alpha = 0.85$. (Choose any sparse example.)

PROBLEM 12.2

Prove that if $W \in \mathbb{R}^{d \times d}$ has all singular values < 1, a deep linear network $\mathbf{y} = W^L \mathbf{x}$ suffers vanishing gradients as $L \to \infty$. Relate this to the need for residual connections.

RECAP CARD

- Every ML algorithm you love is "some matrix factorisation plus a loss".
- Sparse eigen-methods power the web (PageRank).
- Low-rank completion powers recommenders.
- Neural nets are dense-matrix products at scale.

Appendix A · Answer Key

Worked solutions for the Problems in each chapter. MCQ answers are given inline at the question.

Chapter 1

1.1 (i) $(4, -2, -2)$; (ii) $3 + 0 - 8 = -5$; (iii) $\cos\theta = -5/(3 \cdot 5) = -1/3,\ \theta \approx 109.5^\circ$; (iv) $\hat{\mathbf{a}} = (1/3, -2/3, 2/3)$.

1.2 Score $= 2(0.5) + (-1)(-1.2) + 0.5(3) - 0.25 = 1 + 1.2 + 1.5 - 0.25 = 3.45 > 0 \Rightarrow \hat{y} = 1$.

Chapter 2

2.1 $AB = \begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix},\ BA = \begin{pmatrix} 3 & 4 \\ 1 & 2 \end{pmatrix}$; indeed $(AB)^\top = B^\top A^\top$.

2.2 Z is 1000×10. Scalar mults: $1000 \cdot 10 \cdot 50 = 5 \times 10^5$.

Chapter 3

3.1 $x = 2,\ y = 1,\ z = 1$ (unique).

3.2 Unique if $k \neq 2$; infinitely many if $k = 2$ (identical equations); never "no solution" given the matching RHS.

Chapter 4

4.1 $= 2\det\begin{pmatrix} -1 & 4 \\ 2 & 0 \end{pmatrix} - 0 + \det\begin{pmatrix} 1 & 3 \\ -1 & 4 \end{pmatrix} = 2(-8) + 7 = -9$.

4.2 Upper-triangular; $\det(A - \lambda I) = (3 - \lambda)(2 - \lambda) = 0$, so $\lambda = 2, 3$.

Chapter 5

5.1 Rank 2. Column-space basis: columns 1 and 2 of A. Null-space basis: $(-1, -1, 1)^\top$.

5.2 $\mathbf{u} \in C(A^\top) \Rightarrow \mathbf{u} = A^\top \mathbf{z}$. Then $\mathbf{u}^\top \mathbf{v} = \mathbf{z}^\top A \mathbf{v} = \mathbf{z}^\top \mathbf{0} = 0$.

Chapter 6

6.1 $R_{90} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$; reflection $F_y = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}$. Composition $F_y R_{90} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$. Applied to $(1, 0)^\top$: $(0, 1)^\top$.

6.2 Matrix $\begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & -1 \end{pmatrix}$. $\ker = \operatorname{span}\{(1, -1, -1)^\top\}$; image $= \mathbb{R}^2$.

Chapter 7

7.1 $A^\top A = \begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix}$, $A^\top \mathbf{y} = (6,0)^\top$. $\hat{\boldsymbol{\beta}} = (5,-3)^\top$. Residual $(1,-2,1)^\top$ is orthogonal to both columns of A.

7.2 $\mathbf{q}_1 = (1,1,0)/\sqrt{2}$; $\mathbf{u}_2 = (1,0,1) - \frac{1}{2}(1,1,0) = (1/2,-1/2,1)$; $\mathbf{q}_2 = (1,-1,2)/\sqrt{6}$.

Chapter 8

8.1 $\lambda = 3$ with $\mathbf{v} = (1,1)^\top/\sqrt{2}$; $\lambda = 1$ with $\mathbf{v} = (1,-1)^\top/\sqrt{2}$. The eigenvectors are orthogonal (spectral theorem).

8.2 $\det(A - I) = 0$; eigenvector $(2,1)^\top$ normalised to $(2/3, 1/3)^\top$ — stationary distribution.

Chapter 9

9.1 $A^\top A = \begin{pmatrix} 25 & 20 \\ 20 & 25 \end{pmatrix}$, eigenvalues $45, 5$; singular values $\approx 6.71,\ 2.24$. Matching V, U completes the SVD.

9.2 Variance $= \sigma^2 / \sum \sigma^2 \approx (100, 64, 4, 0.01, 0.0025)/168.01$. Cumulative: $\approx 0.595,\ 0.976,\ 0.999,\ \ldots$ — two PCs capture $\approx 97.6\%$.

Chapter 10

10.1 $L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix}$, $U = \begin{pmatrix} 2 & 1 & 1 \\ 0 & -8 & -2 \\ 0 & 0 & 1 \end{pmatrix}$.

10.2 $L = \begin{pmatrix} \sqrt{2} & 0 \\ 1/\sqrt{2} & \sqrt{5/2} \end{pmatrix}$.

Chapter 11

11.1 $\kappa(A) \approx 4 \times 10^4$ — tiny RHS perturbations blow up 4×10^4 times in the solution.

11.2 Cholesky exploits symmetry: only the lower triangle is touched, halving the work ($\frac{1}{3}n^3$ vs $\frac{2}{3}n^3$).

Chapter 12

12.1 One iteration: $\mathbf{r}_1 = \alpha M \mathbf{r}_0 + (1-\alpha)\frac{1}{n}\mathbf{1}$. For any chosen sparse M the vector converges within ~50 iterations.

12.2 $\|W^L \mathbf{x}\| \le \sigma_{\max}(W)^L \|\mathbf{x}\| \to 0$ exponentially. Residual connections $\mathbf{y} = \mathbf{x} + W\mathbf{x}$ make the effective Jacobian $(I + W)$, keeping singular values near 1.

Appendix B · NumPy Cheatsheet

Operation	NumPy / SciPy
Dot product	`u @ v` or `np.dot(u, v)`
Norm	`np.linalg.norm(v)`
Matrix product	`A @ B`
Transpose	`A.T`
Inverse	`np.linalg.inv(A)` (rarely use)
Solve $A\mathbf{x} = \mathbf{b}$	`np.linalg.solve(A, b)`
Least squares	`np.linalg.lstsq(A, b, rcond=None)`
Rank	`np.linalg.matrix_rank(A)`
Determinant	`np.linalg.det(A)`
Eigenvalues / vectors	`np.linalg.eig(A)` , `eigh(A)` (symmetric)
SVD	`np.linalg.svd(A, full_matrices=False)`
QR	`np.linalg.qr(A)`
LU	`scipy.linalg.lu(A)`
Cholesky	`np.linalg.cholesky(A)`
Condition number	`np.linalg.cond(A)`
Pseudoinverse	`np.linalg.pinv(A)`
Sparse CG	`scipy.sparse.linalg.cg`

Appendix C · Glossary

Basis

An independent spanning set for a subspace.

Condition number

Ratio $\sigma_{\max}/\sigma_{\min}$; measures numerical sensitivity.

Determinant

Signed volume factor; zero iff singular.

Eigenvalue

Scalar λ with $A\mathbf{v} = \lambda\mathbf{v}$ for some non-zero $\mathbf{v}$.

Frobenius norm

$\|A\|_F = \sqrt{\sum a_{ij}^2} = \sqrt{\sum \sigma_i^2}$.

Null space

$\{\mathbf{x} : A\mathbf{x} = \mathbf{0}\}$.

Orthogonal matrix

Square Q with $Q^\top Q = I$; a rotation / reflection.

Positive definite

Symmetric A with $\mathbf{x}^\top A\mathbf{x} > 0$ for all $\mathbf{x} \neq \mathbf{0}$.

Rank

Dimension of the column (equivalently row) space.

Singular value

Non-negative entry of Σ in the SVD.

Span

Set of all linear combinations of a collection of vectors.

Trace

$\operatorname{tr}(A) = \sum a_{ii} = \sum \lambda_i$ (eigenvalues).

Appendix D · References & Further Reading

1. Gilbert Strang — *Introduction to Linear Algebra* (6th ed.), Wellesley-Cambridge Press.
2. Gilbert Strang — *Linear Algebra and Learning from Data*, Wellesley-Cambridge Press, 2019.
3. Trefethen & Bau — *Numerical Linear Algebra*, SIAM, 1997.
4. Deisenroth, Faisal & Ong — *Mathematics for Machine Learning*, Cambridge Univ. Press, 2020 (free PDF at mml-book.com).
5. Golub & Van Loan — *Matrix Computations* (4th ed.), Johns Hopkins, 2013.
6. 3Blue1Brown — *Essence of Linear Algebra* video series, YouTube.
7. MIT OpenCourseWare 18.06 — Prof. Strang's lectures.